

SemNav: Semantic Navigation for Repository-Level Issue Localization

Yunxiang Wei¹, Zhenyu Lei², Jundong Li²

¹Independent Researcher

²University of Virginia

yunxiangwei7@gmail.com

vjd5zr@virginia.edu, jundong@virginia.edu

Abstract

Repository-level issue localization aims to identify and rank the files and functions relevant to resolving a reported issue. LLM agents approach this task iteratively: they identify a set of potentially relevant locations, inspect the corresponding code, and revise their judgments about these candidates as new evidence is acquired. Existing environments, however, provide limited support for this loop: agents must search for unresolved relation targets, reconstruct entity semantics from raw source code, and revise candidates without evidential basis. To address these limitations, we present SemNav, a framework that leverages deterministic retrieval to seed a broad candidate set and an LLM agent to continually refine that set, thereby combining initial coverage with evidence-guided revision. SemNav supports this process through three key components. A Semantic Navigation Graph resolves program relations on demand through a language server, enabling direct navigation to related entities across files. Issue-conditioned Semantic Cards provide compact, source-grounded interpretations of each entity’s role and relevance to the issue. A persistent Candidate Workspace records each candidate together with its evidential basis, enabling grounded verification, revision, and ranking. Across SWE-bench Lite and PLocBench, SemNav outperforms existing baselines, improving File Hit@10 from 68.33% to 82.67% with Gemma 4B. Component ablations and trajectory analysis support the complementary roles of all three components, while Semantic Cards reduce working-context load by 48.2% relative to full-source reading. SemNav further ranks first on all seven evidence-quality metrics on SWE-Explore and improves downstream issue resolution from 44.00% to 52.33%.

Introduction

Coding agents are increasingly deployed on real-world software repositories: given a natural-language issue report and a codebase, an agent explores the repository and returns a patch intended to resolve the issue. These repositories often contain far more code than an agent can inspect, while only a small portion is relevant to any single issue, so the agent must first narrow its exploration to a set of potentially relevant locations. *Issue Localization* formalizes this step as the task of identifying and ranking the candidate files and functions that may be involved in resolving the issue. These locations determine which parts of the codebase are available for the agent’s subsequent reasoning and editing, and strongly influence downstream issue resolution (Zhang et al.

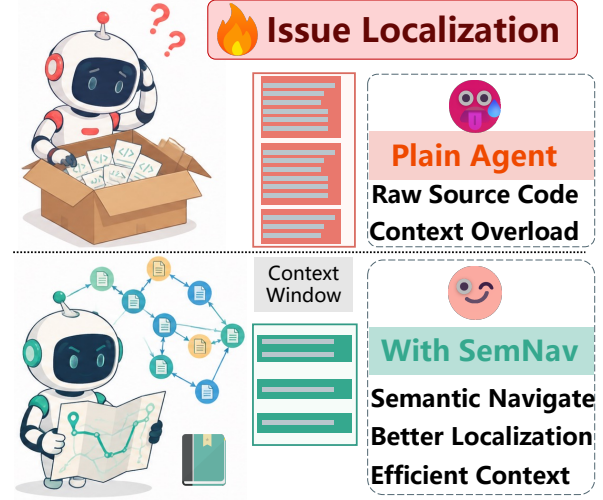


Figure 1: Issue localization with raw-source browsing versus SemNav’s semantic navigation and focused repository evidence.

2026a; Sutawika et al. 2026). Effective localization therefore requires high recall over the locations needed for the fix, while keeping the amount of irrelevant code the agent must inspect small. Figure 1 previews the contrast between raw-source browsing and SemNav’s focused repository investigation.

Existing localization methods differ mainly in how they organize this exploration. Workflow-driven methods apply predefined retrieval, filtering, and reranking stages, typically narrowing from repositories to files and then to functions (Xia et al. 2025; Xie et al. 2025). Such pipelines offer broad coverage and a predictable procedure, but pruning is one-way: a location discarded at an early stage cannot be recovered later. Agent-driven methods instead let an LLM decide what to search, read, or navigate next, through general-purpose commands or specialized repository tools (Zhang et al. 2024; Chen et al. 2025; Zhang et al. 2026b). They can acquire repository evidence adaptively, but the locations under consideration and their supporting rationales remain tied to a single path-dependent reasoning trajectory. To retain information beyond the trajectory, recent agent-based methods

maintain explicit external state, most commonly a running set of candidate locations (Jiang et al. 2025; Yu et al. 2025; Ma et al. 2025a). This makes the two designs complementary: a workflow can supply broad initial coverage, while an agent revises that set through continued repository interaction. Localization then becomes a loop in which the current candidates guide evidence acquisition, and newly acquired evidence guides candidate refinement.

We observe that this localization process involves three core actions: acquiring repository evidence, interpreting its relevance to the issue, and revising localization candidates as evidence accumulates. Existing localization environments face unique challenges in supporting these actions. *(i) Ambiguous relation targets.* Syntax-based graphs can identify program relations but cannot always resolve the repository entities they reach; `obj.run()`, for example, indicates a call without determining which implementation of `run` is invoked. The agent must therefore search among possible destinations instead of directly navigating connected evidence across files, making evidence acquisition costly. *(ii) Implicit entity semantics.* Although source code contains the information needed to judge relevance, an entity’s meaning is often distributed across implementation details rather than stated in compact semantic representation. An agent that consumes raw code must spend its limited context and reasoning capacity reconstructing that purpose before it can interpret the entity against the issue. *(iii) Underspecified candidate state.* A location-only set records which entities are under consideration but not the semantic and evidential basis of its current localization judgment, making successive updates difficult to ground and compare. Together, these limitations place the burden of turning repository observations into evidence-grounded localization decisions on the agent’s limited reasoning context.

Software-engineering practice suggests how this burden can be shifted from the reasoner to its environment. When developers encounter unfamiliar code, IDE operations such as go-to-definition take them directly to related implementations, while declarations and documentation help them understand an entity before inspecting its details. Applied to iterative localization, this practice motivates an environment that makes relations directly navigable and entity meaning readily accessible, while preserving candidate judgments as evidence accumulates.

Motivated by this insight, we propose **SemNav**, an agent-based framework for candidate-centered issue localization. Given an issue and a repository, SemNav leverages deterministic retrieval to obtain a broad set of candidates and then employs an LLM-based Localization Agent to investigate the repository and continually revise them. As illustrated in Figure 2, SemNav equips the Agent with six tools for navigating repository evidence and revising candidates, supported by three key components. *(i) Semantic Navigation Graph (SNG).* The SNG organizes repository entities and resolves requested relation targets online through a language server, allowing the Agent to follow key cross-file relations rather than search among ambiguous destinations. *(ii) Semantic Cards.* On-demand, source-grounded Semantic Cards provide the Agent with a compact entity-level interpre-

tation, reducing the context and reasoning required to recover meaning from implementation details alone. *(iii) Candidate Workspace.* The Candidate Workspace maintains the Agent’s current assessment of each candidate together with its evidential basis, allowing candidate revision to remain explicit and grounded as new evidence arrives. Together, these components enable semantic navigation by integrating repository semantics into both the Agent’s exploration and its continual revision of candidate judgments.

Experiments on SWE-bench Lite (Jimenez et al. 2024) and PLocBench (Zhang et al. 2025) show that SemNav outperforms existing methods across benchmarks and models. With Gemma4 4B, SemNav raises File Hit@10 on SWE-bench Lite from 68.33% to 82.67%. Component ablations and trajectory analysis further verify the complementary roles of our designed three components: the online relation resolution and the Semantic Cards improve localization accuracy, the Candidate Workspace improves both top-ranked accuracy and candidate coverage during iterative revision, and the Semantic Cards reduce the context required to interpret repository entities. On SWE-Explore (Zhang et al. 2026a), SemNav ranks first on all seven evidence-quality metrics, raising line-level precision from 53.35% to 79.99%. Finally, when the localized evidence is used as the repository context for program repair, SemNav achieves the highest patch application and issue-resolution rates, increasing the resolution rate from 44.00% to 52.33% with the Qwen3.5 35B repair backend.

Methodology

Background

An abstract syntax tree (AST) represents source code through grammatical constructs such as declarations, expressions, and statements. Its hierarchy records how these constructs are nested, while source spans map them back to their locations in a file. AST traversal can therefore identify repository entities and locate syntactic occurrences of calls, imports, references, and inheritance. However, syntax records the form and location of an occurrence rather than necessarily determining the project-wide binding of the symbol it contains.

A language server performs project-aware static analysis and exposes position-indexed code-intelligence operations. Go-to-definition maps a symbol occurrence to its resolved definition when available, whereas find-references returns source positions associated with a given definition. Our Python implementation uses Pyright¹ for these operations. Because their results depend on statically available project information, language-server queries may remain unresolved for dynamic behavior or insufficiently constrained symbols.

Overview

Given a repository snapshot $\mathcal{R}$, a natural-language issue q , and a target granularity $\tau \in \{\text{file}, \text{func}\}$, repository-level issue localization aims to return an ordered list of K code entities implicated in resolving the issue:

$$\hat{\mathbf{L}}_q^\tau = \langle v_1, \dots, v_K \rangle, \quad v_i \in \mathcal{V}_\tau. \quad (1)$$

¹<https://microsoft.github.io/pyright/>

Offline Graph Initialization

1 Semantic Navigation Graph

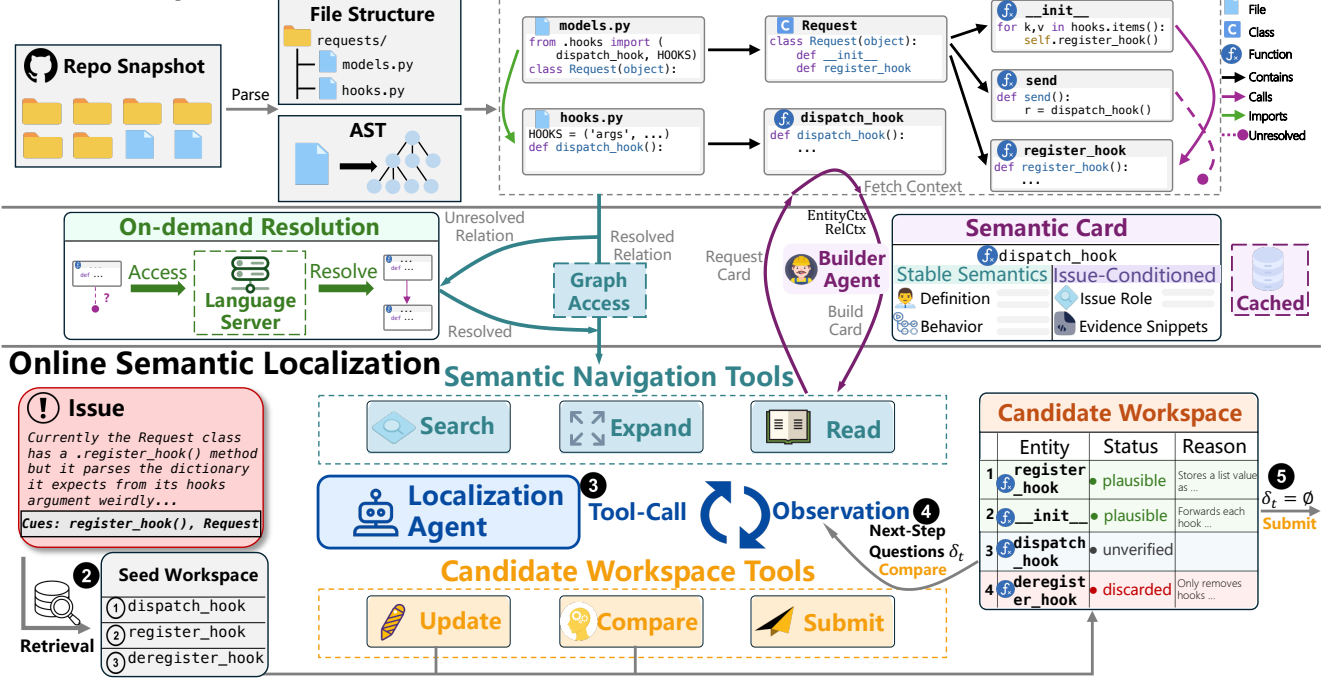


Figure 2: Overview of SemNav.

Here, $\mathcal{V}_\tau$ is the set of file or function targets; class entities are retained as intermediate navigation structures. For each issue, a lightweight rule-based parser identifies explicit code mentions in q and normalizes them into a set of source-facing cues, denoted by $\phi(q)$, such as file paths, identifiers, and API calls.

The SemNav pipeline consists of two stages. In *offline graph initialization*, SemNav uses AST analysis to initialize the Semantic Navigation Graph (SNG) and builds an entity-level retrieval index, retaining relations whose targets remain unresolved. In *online semantic localization*, a deterministic retrieval pass uses q and $\phi(q)$ to initialize the Candidate Workspace W_0 . An LLM-based *Localization Agent* then investigates the repository through the SNG while maintaining and revising localization hypotheses in the Workspace. During this investigation, SemNav queries the language server to resolve relation targets on demand and updates the SNG with the successfully resolved relations.

We equip the Localization Agent with six tools organized into two groups. The first group comprises the *semantic navigation tools* SEARCH, EXPAND, and READ, which allow the Agent to locate entities, access program relations through the SNG, and inspect source code or on-demand Semantic Cards. The second comprises the *Candidate Workspace tools* UPDATE, COMPARE, and SUBMIT, which record candidate judgments, compare plausible candidates, and produce the final ranking. The navigation tools return observations without modifying the Workspace; UPDATE changes candidate records, while COMPARE may reorder them before submission. Starting from W_0 , the Agent alternates be-

tween semantic navigation and Workspace updates. When invoked, COMPARE either returns a focused verification question δ_t that guides further investigation or confirms the current Workspace, after which SUBMIT returns the ranked locations in Eq. (1). The following subsections detail the SNG, semantic navigation with Semantic Cards, and the Candidate Workspace, respectively.

Semantic Navigation Graph

Given a repository $\mathcal{R}$, SemNav represents its current SNG as

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \tilde{\mathcal{E}}), \quad \mathcal{V} = \mathcal{V}_{\text{file}} \cup \mathcal{V}_{\text{class}} \cup \mathcal{V}_{\text{func}}, \quad (2)$$

where $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{T} \times \mathcal{V} \times \mathcal{L}$ contains resolved edges and $\tilde{\mathcal{E}} \subseteq \mathcal{V} \times \mathcal{T} \times \mathcal{V} \times \mathcal{L}$ contains unresolved edges. Here, $\mathcal{L}$ is the set of source locations. A resolved edge $e = (u, r, v, \ell)$ represents relation r from entity u to entity v at location ℓ . An unresolved edge $\tilde{e} = (u, r, \ell)$ leaves its target to online resolution. Methods are represented as function entities, and each entity is identified by its qualified name and source span. For example, `func:pkg/core.py::Parser.parse` identifies the method `Parser.parse` in `pkg/core.py`.

The relation set $\mathcal{T}$ contains CONTAINS, CALLS, IMPORTS, INHERITS, and REFERENCES. CONTAINS connects a file or class to a directly declared entity; CALLS connects a caller to its callee; IMPORTS connects an importing entity to an imported repository entity; and INHERITS connects a subclass to its base class. REFERENCES covers the remaining symbol uses that are not categorized as calls, imports, or inheritance. All relations are directed from the source entity to the target entity.

Offline graph initialization constructs

$$\mathcal{G}_0 = (\mathcal{V}, \mathcal{E}_0, \tilde{\mathcal{E}}_0) = \text{ASTInit}(\mathcal{R}). \quad (3)$$

AST analysis extracts repository entities and identifies the source positions of relevant syntactic constructs, which we call *relation sites*. Relations whose targets are syntactically determined form $\mathcal{E}_0$, while relation sites with unresolved targets are recorded in $\tilde{\mathcal{E}}_0$.

During online localization, SemNav resolves the relations requested for an entity through

$$\mathcal{G}' = \text{Access}(\mathcal{G}; v, r, d), \quad d \in \{+, -\}. \quad (4)$$

ACCESS retrieves resolved relations from $\mathcal{E}$ and queries the language server to resolve the relevant relation sites recorded in $\tilde{\mathcal{E}}$. For forward access ($d = +$), it applies go-to-definition at each outgoing unresolved edge $\tilde{e} = (v, r, \ell)$; a successfully resolved target v' produces $e = (v, r, v', \ell)$. For reverse access ($d = -$), it applies find-references to v and matches the returned relation sites to unresolved edges $\tilde{e} = (u, r, \ell)$, producing $e = (u, r, v, \ell)$. Successfully resolved edges are added to $\mathcal{E}$, while sites without a resolved repository target remain in $\tilde{\mathcal{E}}$, yielding the updated SNG $\mathcal{G}'$.

Semantic Navigation and Semantic Cards

SemNav exposes the SNG through three evidence-facing tools. SEARCH establishes issue-relevant entry points, EXPAND follows typed relations between entities, and READ presents selected entities at structural, source, and semantic levels. Whenever a tool requires program relations, SemNav applies ACCESS to resolve the relevant edges and operates on the updated SNG $\mathcal{G}'$.

Search. For each entity v , SemNav indexes a source-grounded textual representation $D(v)$ formed from its qualified name, declaration, documentation, and implementation text. Given an Agent-generated query x , issue cues $\phi(q)$, and an optional entity or path scope σ , SEARCH first computes BM25 lexical relevance (Robertson and Zaragoza 2009) and selects an anchor set:

$$b(v \mid x, q) = \alpha \text{BM25}(x, D(v)) + \beta \sum_{c \in \phi(q)} \mu(c, v), \quad (5)$$

$$\mathcal{A} = \text{Top}_L \underset{u \in \mathcal{V}_\sigma}{b(u \mid x, q)}.$$

Here, the matcher $\mu(c, v) \in \{0, 1\}$ indicates whether a normalized cue c has an exact keyword or source-location match in v . Search then reranks entities using the resolved neighborhoods of the anchors in the updated SNG $\mathcal{G}'$:

$$\text{Search}(x, \sigma; q) = \text{Top}_M \underset{v \in \mathcal{V}_\sigma}{\left[b(v \mid x, q) + \gamma \sum_{u \in \mathcal{A}} \mathbf{1}[u \sim_{\mathcal{G}'} v] \right]}. \quad (6)$$

We write $u \sim_{\mathcal{G}'} v$ when a resolved SNG edge connects u and v in either direction. SEARCH returns the top M entities together with the textual, cue, and relation evidence supporting each match.

Expand. EXPAND performs typed one-hop navigation over the SNG. For relation type $r \in \mathcal{T}$, the outgoing and incoming neighborhoods in a current graph state are

$$\begin{aligned} \mathcal{N}_r^+(v; \mathcal{G}) &= \{u \mid \exists \ell : (v, r, u, \ell) \in \mathcal{E}\}, \\ \mathcal{N}_r^-(v; \mathcal{G}) &= \{u \mid \exists \ell : (u, r, v, \ell) \in \mathcal{E}\}. \end{aligned} \quad (7)$$

$$\text{Expand}(v, r, d) = \mathcal{N}_r^d(v; \mathcal{G}'), \quad d \in \{+, -\}. \quad (8)$$

EXPAND returns the resulting neighbors together with their relation types, directions, and source locations.

Read. READ presents a selected entity at three evidence levels:

$$\text{Read}(v, m; q) = \begin{cases} \text{Summary}(v), & m = \text{summary}, \\ \text{Code}(v), & m = \text{code}, \\ C_q(v), & m = \text{card}. \end{cases} \quad (9)$$

The issue q conditions only the card mode. The summary mode is a deterministic projection of the indexed declaration, containment structure, and immediate relation profile of v , providing a compact structural orientation. The code mode returns the original source span without accessing additional relations, providing the richest and most authoritative evidence about the entity. While large spans consume substantial context, they may be truncated under the observation budget and require the Localization Agent to reconstruct their semantics. The card mode directly presents an issue-conditioned semantic understanding together with compact source snippets that support it.

Semantic Card. A Semantic Card is a source-grounded interpretation of one repository entity. It records ① the entity's definition and responsibility; ② its operational behavior; ③ its role in the reported issue; and ④ compact source snippets supporting these interpretations. SemNav constructs the Card with a single Builder Agent invocation:

$$\begin{aligned} X_q(v) &= (q, \text{EntityCtx}(v), \text{RelCtx}(v; \mathcal{G}')), \\ C_q(v) &= \text{Builder}(X_q(v)), \end{aligned} \quad (10)$$

where $\mathcal{G}'$ denotes the SNG after accessing the relation families used by the Card. The entity context combines the declaration, implementation, and containment structure of v . The relational context collects the resulting typed one-hop neighborhoods,

$$\text{RelCtx}(v; \mathcal{G}') = \{(u, r, \pm) \mid r \in \mathcal{T}, u \in \mathcal{N}_r^\pm(v; \mathcal{G}')\}, \quad (11)$$

where $+$ and $-$ denote outgoing and incoming relations, respectively. Each compact relation record identifies the neighboring entity together with the edge type and direction; the returned relation evidence also retains its supporting source location. The Builder's dedicated context window holds the target source and relational evidence, while the Localization Agent receives only the compact Card and its supporting snippets.

Cards are constructed on demand and cached by repository snapshot, entity, and issue context, allowing repeated reads to reuse the same semantic interpretation. The Localization Agent consumes $C_q(v)$ as a compact entity-level understanding and can request source code when the investigation requires more detailed implementation evidence.

Table 1: Localization results on SWE-bench Lite and PLocBench. Best and second-best results are bold and underlined.

(a) SWE-bench Lite										
Model	Method	File Hit			Function Hit			Function Recall		
		@1	@5	@10	@1	@5	@10	@1	@5	@10
Gemma 4B	Agentless	36.00	45.67	45.67	4.67	5.00	5.00	2.83	3.11	3.11
	CoSIL	51.67	<u>68.33</u>	<u>68.33</u>	24.00	31.33	31.33	16.40	22.91	22.91
	LocAgent	<u>54.33</u>	62.33	62.33	22.67	<u>42.33</u>	<u>43.00</u>	16.41	<u>31.92</u>	<u>32.92</u>
	mini-swe-agent	49.00	49.33	49.33	<u>25.33</u>	28.00	28.00	<u>22.28</u>	25.11	25.11
	RepoSearcher	42.67	56.33	56.33	1.67	5.67	5.67	1.23	4.51	4.51
	SemNav	58.33	78.00	82.67	34.33	47.67	52.00	28.72	35.40	40.72
Qwen3.5 9B	Agentless	44.00	58.33	58.67	12.67	28.00	31.00	12.00	25.89	29.06
	CoSIL	<u>64.00</u>	80.67	<u>80.67</u>	<u>42.33</u>	58.33	<u>58.33</u>	<u>31.97</u>	43.45	45.84
	LocAgent	59.33	74.33	74.33	24.33	51.00	54.00	22.72	<u>48.47</u>	<u>51.42</u>
	mini-swe-agent	29.33	31.67	31.67	25.33	28.33	28.33	22.72	26.89	26.89
	RepoSearcher	6.00	7.00	7.00	0.67	1.67	1.67	0.50	1.39	1.39
	SemNav	66.00	<u>79.67</u>	83.33	47.00	<u>57.33</u>	59.00	38.58	55.14	55.14
Qwen3.5 35B-A3B	Agentless	65.33	86.33	86.33	21.00	54.33	57.00	19.33	52.00	54.56
	CoSIL	70.67	89.33	<u>89.33</u>	39.00	48.33	48.33	35.28	45.92	45.92
	LocAgent	<u>73.67</u>	80.00	80.00	27.67	<u>58.00</u>	<u>62.00</u>	25.44	<u>55.67</u>	<u>59.50</u>
	mini-swe-agent	68.00	69.00	69.00	<u>53.00</u>	56.00	56.33	<u>47.00</u>	52.89	53.22
	RepoSearcher	70.67	87.33	87.33	7.67	22.00	22.00	6.94	20.06	20.06
	SemNav	78.67	<u>88.33</u>	90.00	63.67	72.67	74.00	49.19	56.66	60.70

(b) PLocBench					
Method	Acc@3	Acc@5	MRR@5	Macro R@5	Micro R@5
Agentless	46.16	47.75	40.48	32.46	22.23
CoSIL	<u>57.73</u>	<u>60.41</u>	50.78	<u>42.07</u>	<u>28.65</u>
LocAgent	37.39	37.64	34.73	25.47	16.90
mini-swe-agent	56.76	56.88	<u>56.55</u>	37.80	21.64
RepoSearcher	54.57	57.13	48.20	39.56	27.02
SemNav	63.09	67.36	57.50	47.45	31.77

Table 2: Evidence acquisition results on SWE-Explore.

Method	P	F1	R@500	nDCG	HitFile	FUH	CE
Agentless	43.34	33.35	21.46	54.88	26.84	58.79	51.20
CoSIL	49.33	<u>50.53</u>	<u>34.39</u>	<u>76.28</u>	<u>39.50</u>	<u>82.57</u>	63.50
LocAgent	53.07	9.02	5.88	55.14	31.94	58.79	59.42
mini-swe-agent	<u>53.35</u>	13.55	9.28	75.77	33.19	77.79	<u>63.98</u>
SemNav	79.99	66.94	37.83	86.96	42.42	86.96	88.32

Candidate Workspace

SemNav organizes the localization process around a persistent *Candidate Workspace*, which maintains the candidates discovered during investigation and their current priority. For a requested target granularity $\tau \in \{\text{file, func}\}$, the Workspace at step t is an ordered list of candidate records:

$$h_t^i = (v_t^i, z_t^i, \eta_t^i), \quad v_t^i \in \mathcal{V}_\tau, \quad W_t = \langle h_t^1, \dots, h_t^{n_t} \rangle. \quad (12)$$

The list order gives the current candidate ranking. Each record h_t^i contains the entity v_t^i , its investigation status z_t^i , and an evidence-grounded rationale η_t^i that cites the observations supporting its current assessment. An *unverified* entity has been retrieved or discovered and awaits further examination, a *plausible* entity is supported by the acquired evidence, and a *discarded* entity has been ruled out.

SemNav initializes the Workspace before the Agent begins its investigation:

$$W_0 = \text{Seed}_\tau(q, \mathcal{G}). \quad (13)$$

The seeding procedure applies the retrieval mechanism of SEARCH using the issue description as a fixed query. It inserts the retrieved entities as unverified records and uses their retrieval ranking as the initial candidate order. The resulting Workspace provides the Agent with a broad starting point and can be extended with candidates discovered during subsequent investigation.

Update. To operate on the Candidate Workspace, SemNav provides the Localization Agent with an explicit UPDATE tool:

$$W_{t+1} = \text{Update}(W_t; v, z, \eta). \quad (14)$$

The Agent uses UPDATE to insert a newly discovered candidate or revise the status and rationale of an existing record. As evidence accumulates, an unverified candidate can be promoted to plausible or marked as discarded. The Agent can

also refine a coarse repository location by adding the concrete target entities reached from it. These updates externalize the Agent’s evolving entity-level judgments in a persistent and revisable candidate state.

Compare. Before submission, SemNav confirms the current Workspace through COMPARE:

$$(\bar{W}_t, \delta_t) = \text{Compare}(q, W_t). \quad (15)$$

COMPARE invokes a dedicated Comparison Agent with a bounded dossier containing the rationale, source code, and Semantic Card of each plausible candidate in W_t . The Comparison Agent jointly evaluates these candidates according to their issue relevance and implementation responsibility and writes their resulting order to $\bar{W}_t$. The reranked plausible candidates lead $\bar{W}_t$, while the unverified candidates retain their maintained order. It then assesses whether the leading candidates are sufficiently supported and distinguished for submission.

When further confirmation is required, δ_t specifies a focused verification question. The Localization Agent uses this question to continue semantic navigation and applies further UPDATE operations to $\bar{W}_t$, producing the next Workspace state W_{t+1} before invoking COMPARE again.

Notably, the Builder and Comparison Agents serve as auxiliary agents with separate context windows. They support semantic construction and candidate comparison without carrying their full working contexts into the Localization Agent’s trajectory.

Submit. The localization loop terminates when COMPARE confirms the current ranking or when the budget is exhausted; we denote the terminal step by T . The Agent then submits the latest compared Workspace:

$$\hat{\mathbf{L}}_q^\tau = \text{Submit}_\tau(\bar{W}_T). \quad (16)$$

SUBMIT first returns plausible candidates in the confirmed order and, when fewer than K are available, fills the remaining positions with the highest-ranked unverified candidates. Discarded candidates are excluded. This process transforms broad deterministic retrieval into an evidence-grounded ranking that remains revisable throughout the investigation.

Experiments

Experimental Setup

We evaluate SemNav on three benchmarks. SWE-bench Lite contains 300 issues and evaluates ranked file- and function-level localization (Jimenez et al. 2024). PLocBench contains 821 issues and evaluates ranked file localization (Zhang et al. 2025). We further use 368 SWE-Explore instances to evaluate the source evidence returned by each method beyond conventional localization targets (Zhang et al. 2026a).

For the cross-benchmark comparison, all methods use the same *gemma-4-e4b-it* model (Gemma 4B). On SWE-bench Lite, we also evaluate every method with *Qwen3.5 9B* and *Qwen3.5 35B-A3B* to test whether the results hold across model families and scales. We compare SemNav with workflow-driven and agent-driven baselines, including Agentless (Xia et al. 2025), CoSIL (Jiang et al. 2025), LocAgent (Chen et al. 2025), mini-swe-agent (SWE-Agent Team 2024), and RepoSearcher (Ma et al. 2025b).

On SWE-bench Lite, Hit@ K measures whether the first K predictions contain a gold entity, while Function Recall@ K measures coverage of all gold functions. On PLocBench, Acc@ K measures whether the Top- K list contains a gold file, MRR@5 rewards earlier first hits, and Macro/Micro Recall@5 measure gold-file coverage. On SWE-Explore, P and F1 measure line-level precision and F1; Recall@500 and nDCG@500 evaluate evidence coverage and ranking within a 500-line budget; HitFile measures coverage of core files; First Useful Hit (FUH) rewards earlier useful evidence; and Context Efficiency (CE) measures the fraction of predicted lines that belong to core or optional context. All metrics are reported as percentages, and higher values are better.

Overall Localization Performance

SemNav achieves the strongest overall localization performance on both benchmarks. On SWE-bench Lite, it ranks first in 24 of the 27 model-metric settings and second in the remaining three (Table 1, Panel (a)). The gains hold with both Gemma and Qwen and at both Qwen model scales. They are most consistent at the function level, where SemNav improves both Function Hit and Function Recall with all three models. For example, with Gemma 4B, SemNav raises Function Recall@10 from 32.92 to 40.72 over the strongest baseline.

The same advantage holds for ranked file localization on PLocBench (Table 1, Panel (b)). SemNav ranks first on all five metrics, improving Acc@5 from 60.41 to 67.36 and Macro Recall@5 from 42.07 to 47.45. The joint gains in accuracy, MRR, and recall show that SemNav improves both the ranking and coverage of gold files.

Takeaway. SemNav improves localization across benchmarks, model families, and model scales.

Component Contributions

We ablate the three main components of SemNav on SWE-bench Lite with Gemma 4B (Figure 3, Panels (a)–(b)). For Semantic Navigation, *w/o Semantic Card* replaces issue-conditioned Cards with the remaining source views, while *w/o Online Resolution* retains only the offline AST-initialized SNG. For the Candidate Workspace, *w/o Seed* starts from an empty Workspace, *w/o Workspace* directly returns the Localization Agent’s predictions, and *Seed Only* returns the deterministic initialization without agent investigation.

Both Semantic Cards and online resolution contribute to localization performance. Removing either component lowers all five metrics while leaving the average output size unchanged. File Hit@10 drops by 6.00 points without Semantic Cards and by 7.00 points without online resolution. These drops come from the information available to the agent rather than from returning fewer candidates.

The Seed provides coverage, while agent investigation improves ranking. Removing the Seed slightly improves Hit@1 but lowers Function Recall@10 by 13.20 points. Seed Only preserves a broad candidate set but ranks its head poorly; the full method improves File Hit@1 by 25.33 points through agent investigation. The Workspace keeps candidates available during this process. Without it, the average output drops from 9.96 to 1.50 functions and Function Recall@10 falls by 19.20 points.

We next measure how Semantic Cards change the context processed by the Localization Agent (Figure 3, Panel (c)). We replay the same SemNav trajectories with the same tool actions and observation order, varying only the observations returned by the READ tool. For each READ action, the three variants return truncated source code, full source code, or the Semantic Card of the entity, respectively. By keeping the trajectories fixed, this setup isolates the effect of context representation from the Agent’s exploration policy.

Semantic Cards reduce the Localization Agent’s working context by 48.2% relative to full-source reading and also reduce its peak context. Building the Cards requires additional tokens, but the total cost remains 25.4% below full-source reading and only 4.8% above source truncation. Semantic Cards therefore improve localization while giving the Localization Agent a more compact working context.

Takeaway. Broad seeding alone is not enough: SemNav uses the Workspace to retain and revise candidates, while Semantic Cards reduce the context needed for these decisions.

Evidence Acquisition Quality

SemNav returns cleaner localization context on SWE-Explore, ranking first on all seven metrics (Table 2). Its largest gains are in precision, F1, and context efficiency: line-level precision rises from 53.35% to 79.99%, while Recall@500 also improves. The higher precision therefore does not come from narrowing evidence coverage. Instead, SemNav removes more irrelevant context while preserving the regions needed for localization. This pattern is consistent with its use of resolved relations and issue-conditioned Semantic Cards to guide localization toward relevant code.

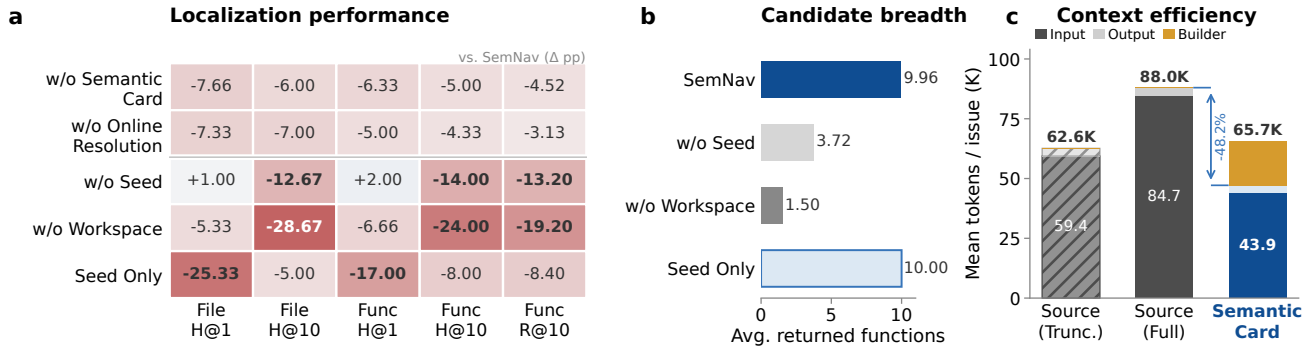


Figure 3: Component analyses on SWE-bench Lite with Gemma 4B.

Table 3: One-shot downstream repair results on SWE-bench Lite. All localization results are produced by Gemma 4B. Green parenthesized values report the fraction of the corresponding oracle resolution rate.

Localization	Gemma 4B Repair		Qwen3.5 35B Repair	
	Applied	Resolved	Applied	Resolved
Agentless	62.67	14.00 (35.9%)	75.33	30.67 (45.8%)
CoSIL	73.00	21.67 (55.6%)	85.00	42.00 (62.7%)
LocAgent	75.33	23.33 (59.8%)	86.67	44.00 (65.7%)
mini-swe-agent	72.00	19.67 (50.4%)	83.67	40.00 (59.7%)
RepoSearcher	65.00	15.33 (39.3%)	77.00	33.00 (49.3%)
SemNav	83.00	27.67 (70.9%)	91.33	52.33 (78.1%)
Oracle source files	90.00	39.00 (100%)	95.00	67.00 (100%)

Takeaway. SemNav improves evidence quality mainly by reducing irrelevant context, not by reducing coverage.

Downstream Repair Utility

To test whether the localization gains transfer to issue resolution, we use the context produced by each Gemma 4B localizer as the only repository context available to two fixed repair models, Gemma 4B and Qwen3.5 35B-A3B. We hold the repair prompt, generation budget, patch validation, and SWE-bench Lite test harness fixed across localization methods. Each model makes one deterministic repair attempt. Applied measures whether a generated patch can be applied to the repository, while Resolved requires it to pass the benchmark evaluation.

SemNav provides the best non-oracle context for both repair models (Table 3). It improves the resolved rate over the strongest baseline by 4.33 points with Gemma 4B and by 8.33 points with Qwen3.5 35B-A3B, while also achieving the highest patch application rate. The benefit with both models shows that SemNav’s localization gains transfer across repair backends. The larger gain with Qwen suggests that a stronger repair model can make effective use of the localized context.

SemNav remains below oracle context with both repair models, showing that better localization can further improve downstream repair. However, oracle context does not resolve every issue, so patch generation remains a separate source of

error.

Takeaway. SemNav’s localization gains translate into better repair performance across both backends.

Related Work

Repository-Level Issue Localization

Repository-level issue localization follows either predefined workflows or agent-directed search. Workflow methods retrieve, filter, and rerank locations: Agentless localizes hierarchically from repository structure to fine-grained entities, SWE-Fixer combines lexical retrieval with learned localization, SweRank applies task-specific retrieval and listwise reranking, and CoSIL searches module- and function-level call graphs in stages (Xia et al. 2025; Xie et al. 2025; Reddy et al. 2026; Jiang et al. 2025). Agent-driven methods let an LLM acquire evidence through AST search, repository-graph traversal, action scheduling, language-server navigation, or shell tools (Zhang et al. 2024; Chen et al. 2025; Yu et al. 2025; Zhang et al. 2026b; Sutawika et al. 2026). Some systems retain candidate locations or repository knowledge outside the reasoning trajectory (Jiang et al. 2025; Yu et al. 2025; Ma et al. 2025a). SemNav combines broad deterministic initialization with evidence-backed agent revision.

Repository Representation and Navigation

Repository representations range from cross-file code fragments (Zhang et al. 2023; Ding et al. 2024; Liang et al. 2024) to graphs of files, classes, functions, and syntax-derived relations. RepoGraph retrieves local definition and reference neighborhoods, CodexGraph supports structured graph queries, LocAgent traverses a heterogeneous entity graph, and CoSIL uses stage-specific call graphs (Ouyang et al. 2025; Liu et al. 2025; Chen et al. 2025; Jiang et al. 2025). RepoNavigator further provides language-server-backed definition navigation (Zhang et al. 2026b), while complementary methods attach natural-language descriptions or conceptual concerns to entities for semantic retrieval (Athale and Vaddina 2025; Wang et al. 2025). Yet syntax-derived relations may leave targets ambiguous, and structural, semantic, and source evidence often remain separate. SemNav unifies on-demand language-server-resolved relations and issue-conditioned Semantic Cards around shared graph

entities, enabling reuse of both evidence types throughout localization.

Conclusion

We propose **SemNav**, a framework for repository-level issue localization that combines broad deterministic retrieval with agent-driven investigation. Through a Semantic Navigation Graph, Semantic Cards, and a Candidate Workspace, SemNav makes repository relations, entity interpretations, and evidence-grounded candidate judgments explicit and revisable throughout localization. Experiments across benchmarks and models demonstrate improved localization accuracy and evidence quality, reduced context requirements for the Localization Agent, and better downstream issue resolution.

References

- Athale, M.; and Vaddina, V. 2025. Knowledge Graph Based Repository-Level Code Generation. In *2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code)*, 169–176.
- Chen, Z.; Tang, R.; Deng, G.; Wu, F.; Wu, J.; Jiang, Z.; Prasanna, V.; Cohan, A.; and Wang, X. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds., *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8697–8727. Vienna, Austria: Association for Computational Linguistics. ISBN 979-8-89176-251-0.
- Ding, Y.; Wang, Z.; Ahmad, W.; Ramanathan, M. K.; Nallapati, R.; Bhatia, P.; Roth, D.; and Xiang, B. 2024. CoCoMIC: Code Completion by Jointly Modeling In-file and Cross-file Context. In Calzolari, N.; Kan, M.-Y.; Hoste, V.; Lenci, A.; Sakti, S.; and Xue, N., eds., *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 3433–3445. Turin, Italy: European Language Resources Association (ELRA) and ICCL.
- Jiang, Z.; Ren, X.; Yan, M.; Jiang, W.; Li, Y.; and Liu, Z. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 3034–3045.
- Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. R. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues? In *The Twelfth International Conference on Learning Representations*.
- Liang, M.; Xie, X.; Zhang, G.; Zheng, X.; Di, P.; wei jiang; Chen, H.; Wang, C.; and Fan, G. 2024. REPOFUSE: Repository-Level Code Completion with Fused Dual Context. arXiv:2402.14323.
- Liu, X.; Lan, B.; Hu, Z.; Liu, Y.; Zhang, Z.; Wang, F.; Shieh, M. Q.; and Zhou, W. 2025. CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases. In Chiruzzo, L.; Ritter, A.; and Wang, L., eds., *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 142–160. Albuquerque, New Mexico: Association for Computational Linguistics. ISBN 979-8-89176-189-6.
- Ma, Y.; Yang, Q.; Cao, R.; Li, B.; Huang, F.; and Li, Y. 2025a. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, FSE Companion '25*, 238–249. New York, NY, USA: Association for Computing Machinery. ISBN 9798400712760.
- Ma, Z.; Peng, C.; Zeng, Q.; Gao, P.; Zou, Y.; and Xie, B. 2025b. Tool-integrated Reinforcement Learning for Repo Deep Search. arXiv:2508.03012.
- Ouyang, S.; Yu, W.; Ma, K.; Xiao, Z.; Zhang, Z.; Jia, M.; Han, J.; Zhang, H.; and Yu, D. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- Reddy, R. G.; Suresh, T.; Doo, J.; Liu, Y.; Nguyen, X. P.; Zhou, Y.; Yavuz, S.; Xiong, C.; Ji, H.; and Joty, S. 2026. SweRank: Software Issue Localization with Code Ranking. arXiv:2505.07849.
- Robertson, S.; and Zaragoza, H. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 4(1-2): 1–174.
- Sutawika, L.; Soni, A. B.; R, B. S. R.; Gandhi, A.; Yassine, T.; Vijayvargiya, S.; Li, Y.; Zhou, X.; Zhang, Y.; Maben, L. M.; and Neubig, G. 2026. CodeScout: An Effective Recipe for Reinforcement Learning of Code Search Agents. arXiv:2603.17829.
- SWE-Agent Team. 2024. mini-swe-agent. Software repository.
- Wang, Y.; Mao, W.; Wang, C.; Zhou, Z.; Zhou, Y.; Zhao, W.; Lou, Y.; and Peng, X. 2025. Extracting Conceptual Knowledge to Locate Software Issues. arXiv:2509.21427.
- Xia, C. S.; Deng, Y.; Dunn, S.; and Zhang, L. 2025. Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng.*, 2(FSE).
- Xie, C.; Li, B.; Gao, C.; Du, H.; Lam, W.; Zou, D.; and Chen, K. 2025. SWE-Fixer: Training Open-Source LLMs for Effective and Efficient GitHub Issue Resolution. In Che, W.; Nabende, J.; Shutova, E.; and Pilehvar, M. T., eds., *Findings of the Association for Computational Linguistics: ACL 2025*, 1123–1139. Vienna, Austria: Association for Computational Linguistics. ISBN 979-8-89176-256-5.
- Yu, Z.; Zhang, H.; Zhao, Y.; Huang, H.; Yao, M.; Ding, K.; and Zhao, J. 2025. OrcaLoca: An LLM Agent Framework for Software Issue Localization. arXiv:2502.00350.
- Zhang, F.; Chen, B.; Zhang, Y.; Keung, J.; Liu, J.; Zan, D.; Mao, Y.; Lou, J.-G.; and Chen, W. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In Bouamor, H.; Pino, J.; and Bali, K., eds., *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2471–2484. Singapore: Association for Computational Linguistics.

Zhang, S.; Wang, Y.; Liang, J.; Shi, Y.; Zeng, W.; Wang, M.; He, S.; Xu, N.; Ye, S.; Cai, K.; and Gu, X. 2026a. SWE-Explore: Benchmarking How Coding Agents Explore Repositories. arXiv:2606.07297.

Zhang, Y.; Ruan, H.; Fan, Z.; and Roychoudhury, A. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ISSTA 2024, 1592–1604. New York, NY, USA: Association for Computing Machinery. ISBN 9798400706127.

Zhang, Z.; Duan, Y.; Zhang, Y.; Xu, Y.; Wang, Z.; Liang, K.; Li, W.; Liang, J.; Xia, D.; Huang, J.; He, J.; and Wu, Y. 2026b. One Tool Is Enough: Reinforcement Learning for Repository-Level LLM Agents. arXiv:2512.20957.

Zhang, Z.; Wang, J.; Yang, Q.; Pan, Y.; Tang, Y.; Li, Y.; Xing, Z.; Zhang, T.; Li, X.; and Zhang, G. 2025. A Benchmark for Localizing Code and Non-Code Issues in Software Projects. arXiv:2509.25242.